DS 203 – Big Data Essentials

Advanced SQL Lab 3 – PL/pgSQL

Marks: 100

Student ID:2025014777 **Student Name: Tetsuya Maeda**

Answer the following questions:

Functions and Control Structures

Q1. Write a PL/pgSQL function called get_film_length_category that accepts a film_id and returns a text label based on the length of the film:

- 'Short' if length < 60 minutes
- 'Medium' if length is between 60 and 120 minutes
- 'Long' if length > 120 minutes

Use IF-ELSIF-ELSE control structure.

Code and Usage:

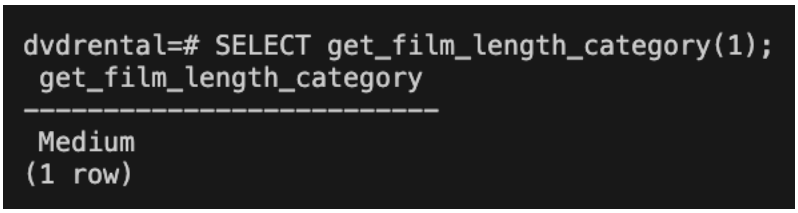
```
CREATE OR REPLACE FUNCTION get_film_length_category(p_film_id 
INT)
RETURNS TEXT AS $$
DECLARE
    v_length INT;
    v_category TEXT;
BEGIN
    -- Get the length of the film from the film table
    SELECT length INTO v_length FROM film WHERE film_id =
p_film_id;

    IF v_length < 60 THEN
        v_category := 'Short';
    ELSIF v_length BETWEEN 60 AND 120 THEN
        v_category := 'Medium';
    ELSE
        v_category := 'Long';
```

```
    END IF;

    RETURN v_category;
END;
$$ LANGUAGE plpgsql;
```

Output Screenshot:



Q2. Write a PL/pgSQL function named `get_customer_status` that accepts a `customer_id` as input and returns the status of the customer. The function should return:

- 'Active' if the customer has rented more than 5 films.
- 'Inactive' if the customer has rented between 1 and 5 films.
- 'New' if the customer has rented no films.

Use the `rental` table to determine the number of films rented by the customer.

Code and Usage:

```
CREATE OR REPLACE FUNCTION get_customer_status(p_customer_id
INT)
RETURNS TEXT AS $$
DECLARE
    v_rental_count INT;
    v_status TEXT;
BEGIN
    -- Count the number of rentals for the given customer
    SELECT COUNT(*) INTO v_rental_count FROM rental WHERE
customer_id = p_customer_id;

    -- Determine the customer's status
    IF v_rental_count > 5 THEN
        v_status := 'Active';
    ELSIF v_rental_count BETWEEN 1 AND 5 THEN
        v_status := 'Inactive';
    ELSE
        v_status := 'New';
    END IF;

    RETURN v_status;
END;
$$ LANGUAGE plpgsql;
```

Output Screenshot:

```
dvdrental=# SELECT get_customer_status(1);
 get_customer_status
---------------------
 Active
(1 row)
```

Functions Only

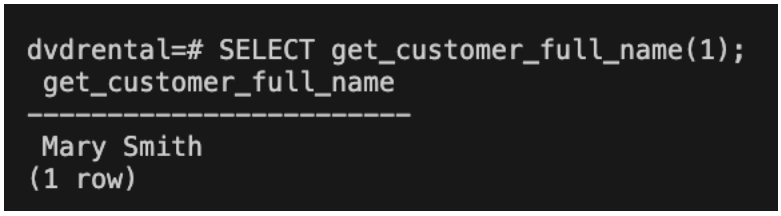
Q3. Create a PL/pgSQL function `get_customer_full_name` that takes `customer_id` as input and returns the customer's full name (concatenation of `first_name` and `last_name` with a space).

Code and Usage:

```
CREATE OR REPLACE FUNCTION get_customer_full_name(p_customer_id
INT)
RETURNS TEXT AS $$
DECLARE
    v_full_name TEXT;
BEGIN
    -- Get the customer's full name
    SELECT first_name || ' ' || last_name INTO v_full_name FROM
customer WHERE customer_id = p_customer_id;

    RETURN v_full_name;
END;
$$ LANGUAGE plpgsql;
```

Output Screenshot:



Procedures and Control Structures

Q4. Write a stored procedure called `print_long_films` that accepts an integer `min_length` and prints all film titles from the `film` table that have a length greater than or equal to `min_length`. Use a FOR loop and RAISE NOTICE to display each title and the length of each film.

Code and Usage:

```
CREATE OR REPLACE PROCEDURE print_long_films(min_length INT)
AS $$
DECLARE
    v_film RECORD;
BEGIN
    -- Loop through films that meet the minimum length
requirement
    FOR v_film IN SELECT title, length FROM film WHERE length >=
min_length LOOP
        -- Raise a notice for each film
        RAISE NOTICE 'Title: %, Length: %', v_film.title,
v_film.length;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Output Screenshot:

```
dvdrental=# CALL print_long_films(180);
NOTICE:  Title: Alley Evolution, Length: 180
NOTICE:  Title: Analyze Hoosiers, Length: 181
NOTICE:  Title: Baked Cleopatra, Length: 182
NOTICE:  Title: Catch Amistad, Length: 183
NOTICE:  Title: Chicago North, Length: 185
NOTICE:  Title: Confidential Interview, Length: 180
NOTICE:  Title: Conspiracy Spirit, Length: 184
NOTICE:  Title: Control Anthem, Length: 185
NOTICE:  Title: Crystal Breaking, Length: 184
NOTICE:  Title: Darn Forrester, Length: 185
NOTICE:  Title: Frontier Cabin, Length: 183
NOTICE:  Title: Gangs Pride, Length: 185
NOTICE:  Title: Haunting Pianist, Length: 181
NOTICE:  Title: Home Pity, Length: 185
NOTICE:  Title: Hotel Happiness, Length: 181
NOTICE:  Title: Impact Aladdin, Length: 180
NOTICE:  Title: Intrigue Worst, Length: 181
NOTICE:  Title: Jacket Frisco, Length: 181
NOTICE:  Title: King Evolution, Length: 184
NOTICE:  Title: Lawless Vision, Length: 181
NOTICE:  Title: Love Suicides, Length: 181
NOTICE:  Title: Mixed Doors, Length: 180
```

Q5. Create a procedure named `show_inventory_by_category` that loops through each film category and prints the category name along with the count of inventory items available in that category (joining `inventory`, `film`, and `category` tables).

Code and Usage:

```
CREATE OR REPLACE PROCEDURE show_inventory_by_category()
AS $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN
        SELECT
            c.name AS category_name,
            COUNT(i.inventory_id) AS inventory_count
        FROM
            category AS c
        JOIN
            film_category AS fc ON c.category_id =
fc.category_id
        JOIN
            inventory i ON fc.film_id = i.film_id
        GROUP BY
            c.name
        ORDER BY
            c.name
    LOOP
        RAISE NOTICE 'Category: %, Inventory Count: %',
r.category_name, r.inventory_count;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Output Screenshot:

Marking Criteria:

For each question:

Correct Code and Usage = 15 Marks

Correct Output Screenshot = 5 Marks

Total Marks per Question: 20

Total Marks: 20 x 5 (questions) = 100 Marks.

Submission Instructions:

Submit the following:

1. Convert this document into PDF and rename it as **"DS 203 - Lab 3 – StudentName.pdf"** then submit in the appropriate submission on Google Classroom.
2. Also, submit the .sql script containing codes for all the questions in one single file with comments and explanation guiding the flow of the script. You can name the script file as **"DS 203 - Lab 3 – StudentName.sql".**